

ADVeRL-ELF: ADversarial ELF Malware Generation using Reinforcement Learning

Akshara Ravi¹, Vivek Chaturvedi¹, Muhammad Shafique²

¹Department of CSE, Indian Institute of Technology, Palakkad, India

²eBRAIN Lab, Division of Engineering, New York University, Abu Dhabi, UAE

Abstract—Deep learning models are now pervasive in the malware detection domain owing to their high accuracy and performance efficiency. However, it is critical to analyze the robustness of these models by introducing adversarial attacks that can expose their vulnerabilities. Nevertheless, adversarial malware generation problem for Linux has not been well-investigated. In this work, we propose a novel reinforcement learning framework, ADeRL-ELF to generate adversarial ELF malware by adding semantic NOPs within the executable region. Experimental results show that ADeRL-ELF achieved an attack success rate of 59.5%. These adversarial malware can be leveraged to harden the Linux based malware detection systems.

Index Terms—adversarial malware, reinforcement learning, Linux ELF, semantic NOPs

I. INTRODUCTION

Malware or malicious software is any software that is introduced intentionally to the computer systems or servers to steal confidential data, disrupt services, or corrupt the system files. Malware has been a serious threat to the security landscape of business enterprises, banking, and e-commerce over the past few decades. Although security experts have developed novel and effective malware detection techniques, malware continues to be a growing concern. According to SonicWall’s 2024 Mid-Year Cyber Threat Report [1], there is a surge in malware attacks (30%) in the first half of 2024. Recent years have witnessed a substantial growth of Linux malware as shown in Fig. 1. One of the main reasons for this explosion is attributed to the proliferation of IoT devices such as smart cameras, smart watches, and smart home appliances. According to IoT Analytics [3], the number of connected IoT devices is expected to reach 18.8 billion by the end of 2024. These devices are the weakest points in a network with limited resources and lapse security exacerbated by constant online connection, default credentials, and open ports. Attackers use these compromised IoT devices as tools for launching cyberattacks such as Distributed Denial of Service (DDoS), cryptomining, and phishing, thus translating to an enormous 107% increase in IoT malware attacks in 2024 [1]. To combat this escalating threat, there is a pressing need for innovative and robust malware detection techniques for Linux.

Apart from the commercial antivirus, state-of-the-art malware detection tools are increasingly exploiting the machine learning (ML) and deep learning (DL) based static and dynamic malware analysis techniques. However, these neural networks are not completely secure and are known to be

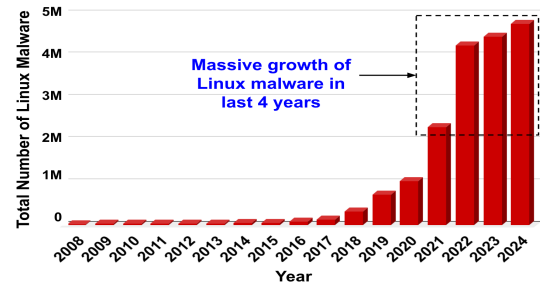


Fig. 1: Growth of Linux malware from 2008 to 2024 [2]

vulnerable to adversarial samples. Adversarial examples (AEs) are maliciously crafted inputs generated by adding small perturbations such that the neural network misclassifies the sample with high confidence. Adversarial attacks were first introduced in the image classification domain [4], where the goal is to make AEs imperceptible to human eyes. Fast Gradient Sign Method (FGSM) [4] and Projected Gradient Descent (PGD) [5] are the most popular adversarial attacks against images. These AEs are used for performing the vulnerability analysis to harden the models. *However, these attacks cannot be directly applied to the malware problem space due to the inverse feature mapping problem [6] illustrated in Fig. 2.*

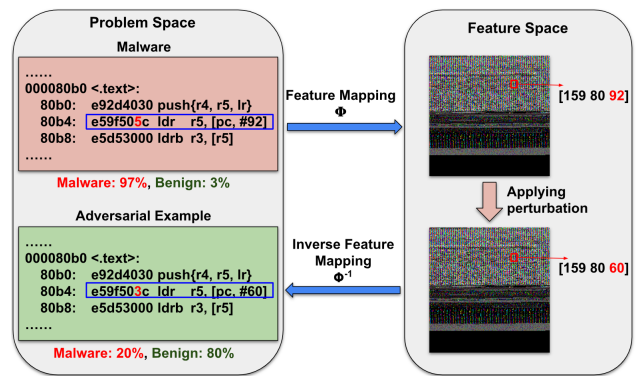


Fig. 2: Illustration of how adding perturbations in the feature space does not translate to adversarial executables with the same functionality in the problem space

In this scenario, malware executables constitute the problem space and images generated from their bytecodes represent the feature space of malware image-based detection models. Other feature spaces for malware include API calls, header features, and opcode sequences. When we apply adversarial attack such as FGSM on these malware images, it will modify the pixel values. For example, the pixel value at the position

bounded by red box is changed from [159 80 92] to [159 80 60]. Here, only the B channel is modified in the feature space. The generated adversarial image looks exactly similar to the original one and successfully evades the malware detection model. Nevertheless, we could see that the inverse feature mapping function ϕ^{-1} is neither invertible nor differentiable [6] i.e., the single byte modification translates to a different instruction (shown in blue boxes) altering the functionality of the executable. Thus, the perturbations added in the feature space should be carefully crafted such that they do not break the executable. Practically, it is better to impose adversarial perturbations in the problem space. A successful adversarial malware should evade the malware detection model and maintain the malicious and executable functionality [7].

Adversarial malware generation (AMG) techniques for Linux-based operating systems are not well explored. In this work, we propose a novel reinforcement learning (RL) framework ADVeRL-ELF for AMG, where the generated AEs satisfy the malicious and executable functionalities. We have chosen a RL approach since it follows a sequential decision-making process that enables us to generate AEs with minimum perturbation [8]. Further, RL can effectively handle the discrete perturbation space of malware detection domain [9]. ADVeRL-ELF works by inserting semantic NOPs (NO Operations) in the `.text` section of Linux executables. *To the best of our knowledge, this work is a pioneering contribution in AMG for IoT binaries with Executable and Linkable Format (ELF) that add perturbations in the executable region.* To guide the evasion process with a minimum number of NOP insertions, we extract the GradCAM region from the malware images.

Our novel contributions are listed below:

- Developed a novel reinforcement learning framework ADVeRL-ELF for generating adversarial examples that can be leveraged to make the Linux based malware detection systems robust against adversarial attacks.
- Focused on IoT malware with Linux ELF format. State-of-the-art works primarily focus on adversarial Windows PE malware generation.
- In contrast to existing works that alter the non-executable regions, we added perturbations in the executable `.text` section of ELF by adding semantic NOPs.
- Demonstrated that a diverse action space can be advantageous against preprocessing filters that easily filter out naive NOPs for instance, 0x90 (NOP) in Windows and 0xe1a00000 (MOV r0, r0) in Linux ARM).
- Achieved an attack success rate of 59.5%.

Paper Organization: Section 2 discusses the background and related work. Section 3 elaborates on the motivational study. Our proposed methodology is discussed in Section 4. Section 5 summarizes the experiments and results. Finally, Section 6 concludes the paper.

II. BACKGROUND AND RELATED WORK

There is an exponential growth of malware in recent years owing to the surge in computational resources and advanced technologies. This sheer increase in the number of malware

places an immense burden on security teams, and hence they opt for automated malware detection techniques using ML/DL. For instance, ELF-Miner [10] and NtMalDetect [11] utilize ELF header features and system call traces, respectively, for detecting malware. Regardless of how smart and effective these solutions are, adversaries may uncover new adversarial attacks to evade these malware detection techniques.

AMG techniques in the literature mainly leverages iteration based on gradient [12] and RL to apply perturbations. COPY-CAT [13] generates adversarial examples using two main configurations; appending the AE generated using techniques such as FGSM, MIM, C&W, DeepFool, and PGD attack and injecting the samples from the target class at the end. RL-based adversarial attacks have recently gained popularity due to the greater potential of RL in the context of black-box attacks [14] [15]. Some of the actions used by RL agents in the literature [16] [17] [18] [19] include appending random bytes at the end, adding random sections, creating a new entry point that immediately jumps to the original entry point, etc. Majority of these attacks alter the non-executable regions of Windows PE. This ensures the executable and malicious functionality of the malware. However, these perturbations are prone to easy detection by a preprocessing filtering that searches these obvious places where perturbations can be hidden. Daniel et al., [20] introduced a RL framework based on Double Q-network to select the optimal positions to add naive NOPs in the `.text` section. However, naive NOPs can be effectively filtered out by preprocessing filters, thereby reducing the effectiveness of the approach. Further, most malware detection models utilize only the first "x" bytes of the executable rendering the attacks that append at the end of the binary meaningless.

Adversarial attacks in the literature primarily focus on Windows PE or Android binaries. This could be due to the availability of open source tools to parse and edit PE files and the misconception that Linux systems are secure. EvilELF [21] applies 5 methods to modify ELF binaries, such as header, debug, and padding alteration, end appendix, and dynamic extension. Xue et al., [22] applied the Proximal Policy Optimization (PPO) RL algorithm to generate adversarial malware by applying 11 actions. These works alter the non-executable regions of ELF binary. In order to fill this gap, we propose an adversarial attack that hides the perturbations in the executable region of the ELF binaries. Towards that, we conducted a motivational study comparing our proposed framework with other state-of-the-art as shown in Table I. From Table I, we can clearly observe that our work is the first adversarial attack that alters the executable region of the ELF binaries.

III. MOTIVATIONAL CASE STUDY

First, we have conducted three preliminary experiments that motivated us to leverage a reinforcement learning approach for adversarial malware generation. We have selected the image-based malware detection approach, as it tends to take less feature extraction time than other static approaches like header features or opcodes [24]. Due to the popularity of residual networks in the image-based malware detection domain [25]

TABLE I: Summary of existing adversarial malware attacks

Work	Technique	Perturbation in executable region ?	Target binary	Executable & malicious ?
FGAM [7]	Gradient	✗	PE	✓
COPYCAT [13]	Gradient	✗	PE, IoT	✗
Daniel et al., [20]	RL	✓	PE	✓
EvilELF [21]	Random bytes	✗	ELF	✓
Xue et al., [22]	RL	✗	ELF	✓
Qiao et al., [23]	Gradient	✗	ELF	✓
ADVeRL-ELF	RL	✓	ELF	✓

[26] owing to its capability in capturing complex image features, we assume the target model to be ResNet18. Input for this model will be the first 150528 bytes of the executable that correspond to a (224,224) RGB image, a standard size for residual networks. We chose the first part of the image since these portions include header regions that are considered imperative by state-of-the-art malware detection models. Experiments in this section were carried out on a sample dataset of 2572 benign and 3000 malware samples. ResNet18 achieved a test accuracy of 96.22% on original malware samples. Next, we discuss the three motivational studies in detail.

A. Applying adversarial attacks in the malware image domain

We have applied popular adversarial attacks such as FGSM and PGD in the image domain. Results are shown in Fig. 3. Here, for both FGSM and PGD, we obtain an attack success rate (ASR) of 100% with $\epsilon = 0.1$, where ϵ bounds the amount of perturbation added to input. Resulting perturbation rate is 62%-79% which is not desirable for our case.

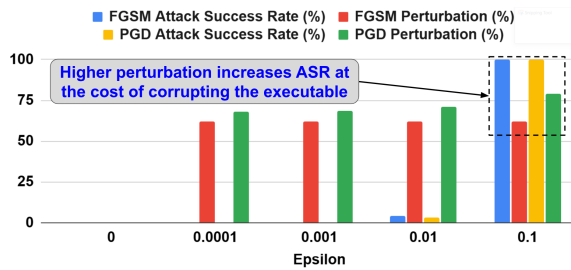


Fig. 3: Attack success rate and perturbation rate when applying FGSM and PGD attacks to malware image domain

B. Adding random benign patches from the dataset

We need to add perturbations such that the malicious functionality is not altered. One approach is to add benign patches to the malware. We need to address 3 questions here.

1) *How to get these patches?*: In this study, we have used GradCAM (Gradient-weighted Class Activation Mapping) to select a (32,32,3) portion of benignware using ResNet18. It uses the gradients of the input image flowing into the final convolutional layer to produce a heat map that highlights the important regions in the image that influence the model decision [27]. Thus, these regions in benignware will incorporate significant benign characteristics.

2) *Where to insert these patches?*: We will insert these benign patches to the GradCAM region of malware so that the malicious probability can reduce at a faster rate. However, if the GradCAM position falls within a section, we cannot add the patch exactly at that point, since it will corrupt the executable. Hence, we found the section where the GradCAM lies and added a new non-executable section of size 3072 bytes (to add (32,32,3) benign patch) before that. Thus, the applied perturbation rate is constant.

3) *How to add these patches?*: First, we randomly selected benign patches for each malware image and repeated the experiment for 10 runs. The average ASR is shown in Table II.

C. Adding random semantic NOPs

Mostly, the GradCAM region falls in the *.text* section of a binary since this section encompasses the actual instructions that determine the malicious or benign behavior and hence mostly affect the model's decision. Still, we cannot add the extracted benign patches or any random perturbation at these positions. In such a case, semantic NOPs, instructions that "do nothing" to the execution flow come to our aid. We have chosen ARM as the use case here, since it is the most predominant architecture for IoT. We have downloaded the list of semantic NOPs for the ARM architecture from the github link [28]. Now, we need to decide on

1) *Where to add these NOPs?*: We again choose the GradCAM region. For a (32,32,3) GradCAM region, we have 32 positions to add semantic NOP sequences of length (32*3=96 bytes). First, we add random semantic NOP sequences at all 32 positions of malware whose GradCAM lies within the *.text* section. Then, we combined benign patch and semantic NOP methods by adding a random benign patch if the GradCAM lies outside the *.text* section. Table II shows the results.

TABLE II: Average attack success rate (ASR) over 10 runs for adding benign patches and semantic NOPs randomly

Approach	ASR (%)
Add a random benign patch for all malware	33.22
Add random semantic NOP sequences of length 96 bytes at 32 positions in the GradCAM region	0
Add random semantic NOP sequences in the Grad-CAM region if <i>.text</i> section else benign patch	22.57

2) *How many NOPs should be added ?*: From Table II, we could see that even after adding random semantic NOP sequences of length 3072 (32*32*3) bytes, ASR was 0%. We have conducted experiments to investigate whether we can increase the attack success rate by increasing the number of semantic NOPs. However, ASR remained around 20%-26% while adding 1 to 32 semantic NOPs for the case where both semantic NOPs and benign patches are added based on the GradCAM position. Thus, randomly adding semantic NOPs even though reduces the malicious probability, it is not enough to change the classification label. Hence, there is a need for an approach that investigates the effect of each semantic NOP on the malicious score of an executable and decide based on its knowledge which NOP to use. Our ADVeRL-ELF framework discussed next solves this problem.

IV. PROPOSED METHODOLOGY

This section presents the overall workflow of the adversarial attack process shown in Fig. 4. It consists of 3 major phases: ADVeRL-ELF framework that generates adversarial ELF malware, ELF manipulation procedure that adds semantic NOPs to the executable, and target malware detector that evaluates the evasion capability of the generated adversarial samples. First, we examine the threat model of the attack.

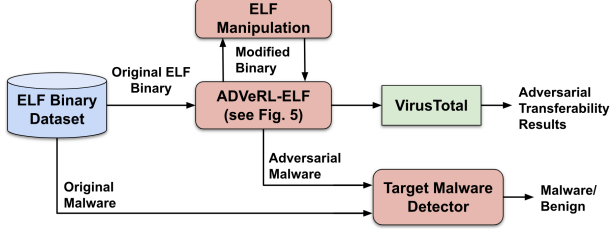


Fig. 4: Overall workflow of the adversarial attack process

A. Threat Model

We assume that the target malware detector uses a visualization-based static analysis approach to detect malware. Specifically, we make the following assumptions.

- 1) **Adversary's Goal:** Generate adversarial ELF malware samples that are misclassified as benign by the target model and satisfy the malicious and executability properties. We are interested in non-targeted attack that transforms malware to benign class and not vice versa.
- 2) **Adversary's Capabilities:** The adversary has access to a set of malicious ELF samples. Further, the adversary has knowledge about the structure of the target malware detector. This helps the adversary estimate the GradCAM region from the last convolutional layer of the model.

B. ADVeRL-ELF Framework

Our proposed adversarial ELF malware generation framework, ADVeRL-ELF shown in Fig. 5, employs the Asynchronous Advantage Actor-Critic (A3C) algorithm [29]. A3C is a policy gradient-based deep reinforcement learning algorithm in which the actor and the critic agents work in tandem to optimize the policy. We will explain the rationale for choosing A3C in Section V.

Here, the environment consists of ELF malicious binaries and the target malware detector, which could be any neural network or antivirus scanners (here, ResNet18). State space of the environment is represented by a (224,224) RGB malware image. We generate RGB images from the bytecode of executables by grouping it into 24 bit sequences of red, green, and blue channels (8 bits each with each byte representing integers in the range [0, 255]). Exploring a high dimensional action space with 183375 NOPs from [28] is cumbersome. We identified that they can be scaled down to 120 NOPs with the below 5 operations that constitute our action space.

$[REV_{xx} \quad REV_{xx}]$, $[REV16_{xx} \quad REV16_{xx}]$, $[ADD_{xx} \quad SUB_{xx}]$, $[SUB_{xx} \quad ADD_{xx}]$, $[EOR_{xx} \quad EOR_{xx}]$

where xx corresponds to the condition codes EQ, NE, LT, GT, LE, GE, CS, CC, MI, PL, VS, VC, HI, LS or AL. The

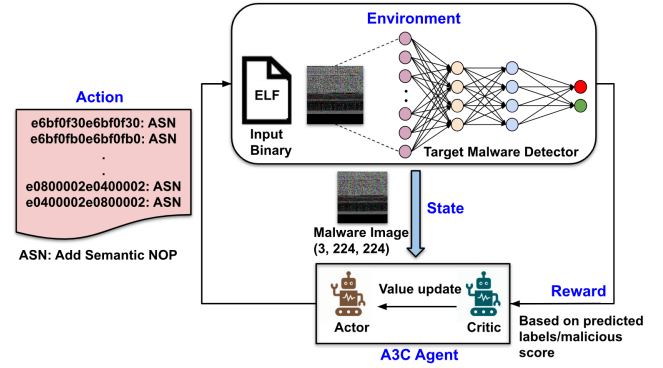


Fig. 5: Proposed ADVeRL-ELF Framework

operands for these operations can be 16 registers R0-R15 in ARM32 and immediate values 0-255.

The actor component of A3C RL agent chooses an 8 byte semantic NOP action from list and inserts it to the GradCAM position in the .text section of the given binary as illustrated in Fig. 6. Then the RGB image corresponding to the modified binary is given as input to ResNet18. Based on the response of the malware detector, we designed two reward functions r_t .

- 1) Based on label: Access to the predicted labels only; 0 (Benignware) and 1 (Malware)

$$r_t = \begin{cases} 10 & \text{if } label_{s_t} = 0 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

- 2) Based on score: Access to the malicious score. Reward is calculated based on malicious threshold as:

$$r_t = \begin{cases} 10 & \text{if } mal_score_{s_t} < mal_threshold \\ (mal_score_{original} - mal_score_{s_t}) & \text{otherwise} \end{cases} \quad (2)$$

Here, if the malicious probability of binary after adding a semantic NOP is decreasing, then we assign a positive reward proportional to the rate of decrease to the action and vice versa. Despite the fact that a high threshold (90%) can increase the ASR, we chose a hard threshold of 50% since a higher threshold will result in higher false negative rate (missed threats) that can be catastrophic.

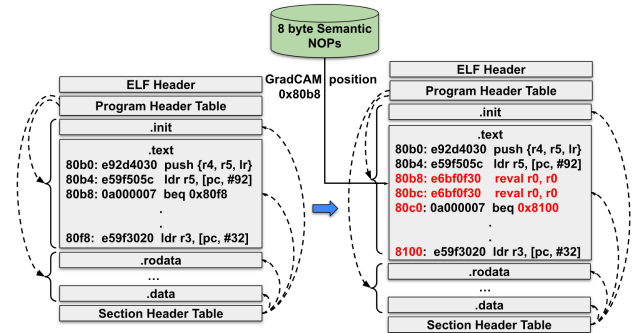


Fig. 6: Overall workflow of adding semantic NOPs

Finally, critic will evaluate how good the selected action was in driving towards our goal i.e., benign based on the reward and state and update the information to actor. Actor will further continue the step by selecting the next action. This process

continues until the malware evades the target malware detector or maximum turns are reached. Algorithm 1 provides a high-level overview of the training procedure for label reward. Here, *IMAGE_FEATURE_EXTRACT()* converts malware bytecode to (224,224) RGB image and *ADD_SEMANTIC_NOP()* edits the ELF executable by adding the semantic NOP corresponding to action a_t using the steps discussed next.

C. ELF Binary Manipulation

In our work, we focus on generating AEs for Linux-based IoT devices. All ARM instructions are 32 bits (4 byte) long, that will expedite our process to identify the instruction boundaries to insert NOPs. We utilize NOPs of length 8 bytes. For adding semantic NOPs in the *.text* section, we leverage *lief* [30], a cross platform library that can parse and edit ELF, PE, and MachO formats. However, the tool does not support some ELF manipulations which were handled by our custom script. In the following, we review the steps to edit ELF executable.

- 1) Parse the ELF executable using *lief*
- 2) Find the GradCAM position within the *.text* section
- 3) Convert the semantic NOP to little endian if EI_DATA field in ELF header is "ELFDATA2LSB"
- 4) Shift the instructions in *.text* section and the later sections and add the 8 byte semantic NOP
- 5) Update the section size of *.text* section and handle alignment properties.
- 6) Update the offset and virtual address of the sections after the *.text* section
- 7) Rebuild the segments
- 8) Update the jump and branch addresses that refer to the instructions after the semantic NOP insertion point
- 9) Finally, write the modified binary

The target malware detector predicts whether the generated adversarial example is malware or benign and thus determines the evasion capability. We further analyze the adversarial transferability of samples against VirusTotal [31]. Next, we will discuss the experiments and the results achieved.

V. EXPERIMENTS AND RESULTS

A. Dataset

In our experiments, we have used the ARM binaries of the IoT malware dataset, IoTPoT [32]. We crawled the GitHub repositories and Linux directories */bin*, */sbin*, */usr/bin*, and */usr/local/bin* of Raspberry Pi and Jetson Orin Nano to prepare the benign dataset.

B. Experimental Setup and Results

We conducted the experiments on an Intel® Core™ i7-10700 CPU @ 2.90GHz × 16 processor running Ubuntu 20.04.6 LTS with 32GB RAM and Python 3.8.10. Lief 0.12.3 was used to edit the ELF executable and chainerrl 0.8.0 was used to build the A3C network. We have further utilized the OpenAI gym framework [33] that enables us to generate custom malware evasion environments for training and evaluating our RL algorithms. First, we trained our target malware detector ResNet18 for 50 epochs and achieved a test accuracy

Algorithm 1 Adversarial Malware Generation

Input: Malicious ELF samples set M , MAX_TURNS , Agent A3C, Action Table A , Target Malware Detector Mal_Detect , True labels y_true

Output: Adversarial malware samples set Adv_M

```

1: Initialize A3C, gym_malenv
2: reward = 0,  $Adv\_M = \phi$ 
3: for  $m$  in  $M$  do
4:    $label_{orig} = \text{PREDICT\_LABEL}(m)$ 
5:   if  $label_{orig} \neq y\_true[m]$  then
6:     "Already Benign"
7:     break
8:   for  $i=1$  to  $MAX\_TURNS$  do
9:     Choose  $a_t \in A$ 
10:     $m_{t+1} = \text{ADD\_SEMANTIC\_NOP}(m_t, a_t)$ 
11:     $label_{t+1} = \text{PREDICT\_LABEL}(m_{t+1})$ 
12:    if  $label_{t+1} \neq y\_true[m]$  then
13:      reward = 10
14:       $Adv\_M = Adv\_M \cup m_{t+1}$ 
15:    else
16:      reward = 0
17: return  $Adv\_M$ 

```

Function PREDICT_LABEL(episode)

```

1:  $s = \text{IMAGE\_FEATURE\_EXTRACT}(\text{episode})$ 
2:  $label = Mal\_Detect.predict(s)$ 
3: return label

```

of 99.69%. However, our ADVeRL-ELF framework focuses on generating adversarial malware, and therefore, we consider the malware samples for our further experiments.

Initial set of experiments were carried out to determine the RL algorithm that works well on our problem space. Deep Q-learning (DQN) and A3C algorithms were initially trained on a sample malware dataset of 100 training samples and 20 test samples. We have applied Adam optimizer. For DQN, we have adopted the LinearDecayEpsilonGreedy explorer and prioritized experience replay. Both the label and score rewards given, respectively, in equations 1 and 2 were considered for our experiments. Results are shown in Fig. 7 for 10000 steps and 100 semantic NOPs. We have chosen the attack success rate metric that depicts the percentage of adversarial malware that successfully evaded the target malware detector out of the total adversarial samples to evaluate our framework.

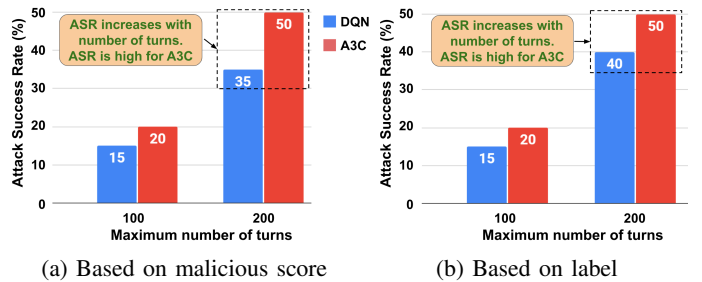


Fig. 7: Attack success rate for DQN and A3C RL algorithms

We have further evaluated the training time of DQN and

A3C. We could see that DQN takes around 43 hours for 50000 steps, whereas A3C takes only 16 hours. We have further conducted experiments by increasing the number of samples and training steps while keeping the number of actions as 100 randomly. Table III summarizes the ASR achieved for 50000 steps and 200 maximum turns.

TABLE III: Attack success rate for increasing samples

Dataset	Reward Function	
	Label (%)	Malicious Score (%)
1102 training and 179 holdout	53.07	54.19
3784 training and 687 holdout	44.69	42.94

From Fig. 7 and Table III, we can infer that the achieved ASR is around 40-55%. When it comes to malware detection, missing out on even a single malware sample can be risky, and hence this evasion rate is problematic for any malware detector. Further, we could see that the ASR is reducing as the number of samples increases. This is because the number of steps must be increased to give all samples enough chance of evasion. Fig. 8 shows the effect of increasing turns on attack success rate. When the number of turns per episode increases, the perturbation increases, resulting in increased ASR.

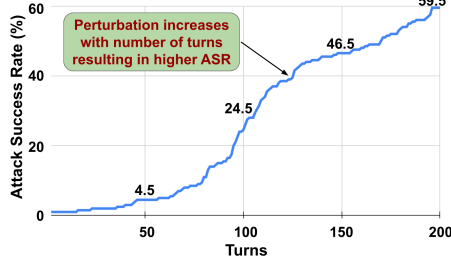


Fig. 8: Number of turns v/s Attack Success Rate

Each turn will add 8 byte semantic NOP and hence the perturbation of a sample for 200 turns will be 1600 (200*8) bytes. This trade-off between ASR and perturbation rate is imperative. A higher perturbation rate can significantly increase the file size that can cause the sample to lose its concealment ability. Thus, we keep the maximum allowed number of turns as 200. We perform our final set of experiments on a dataset of 1000 training and 200 test samples for 1,00,000 steps. Further, we modify the action space to include the 120 semantic NOPs discussed in Section IV-B. Results are shown in Table IV. Out of 200, 119 samples successfully evaded ResNet18.

TABLE IV: Final attack success rate for 200 turns

Reward	Based on label	Based on score
Attack Success Rate (%)	58	59.5

We have submitted these 119 original and adversarial malware to VirusTotal to analyze adversarial transferability against commercial antivirus. Fig. 9 shows the number of antiviruses out of total 64 that can correctly detect maliciousness of the 119 samples on VirusTotal before and after applying our adversarial attack. We could see that on average the number of antiviruses that can detect maliciousness is reduced from 39 to 29 with the maximum reduction in the number of antiviruses from 37 to 8. Finally, we have compared our ADVeRL-ELF framework against the state-of-the-art approach for adversarial

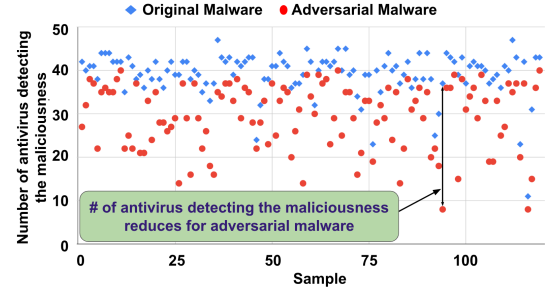


Fig. 9: VirusTotal results on original and adversarial malware

ELF attacks. Table V shows the results. Here, comprehensive disturbance adds arbitrary bytes generated by FGSM attack at the end and middle of an ELF file. Our ADVeRL-ELF framework achieved a better attack success rate of 59.5%.

TABLE V: Comparison with state-of-the-art

Approach	Attack success rate (%)
Comprehensive Disturbance [23]	26.2
ADVeRL-ELF	59.5

Based on the experimental findings, we could determine that ADVeRL-ELF can generate adversarial ELF malware by adding perturbations in the executable region. This is in contrast to the works that add perturbations in the non-executable areas of binary. Thus, ADVeRL-ELF generates AEs that maintain malicious and executable functionality, and these AEs can be used to strengthen existing malware detection systems. Moreover, this framework can be easily extended to state-of-the-art neural networks such as vision transformers and MobileNet and other CPU architectures. For instance, we can utilize the semantic NOPs for MIPS and x86 from the same github repository [28] and ADVeRL-ELF to generate adversarial malware for these architectures.

VI. CONCLUSION AND FUTURE WORK

In this work, we proposed a novel framework ADVeRL-ELF that addresses the existing problem of adversarial ELF malware generation. ADVeRL-ELF generates adversarial malware by adding semantic NOPs in the GradCAM position within the executable region. We have achieved an attack success rate of 59.5% against our target malware detector ResNet18. Further, the average number of virus detection tools that could detect the sample's maliciousness on VirusTotal is reduced from 39 to 29. This work can be further extended to apply binary diversification techniques such as instruction substitution, code displacement, etc. in the executable region. Further, adversarial malware that can evade dynamic malware analysis techniques can be an interesting and challenging avenue to explore.

ACKNOWLEDGEMENTS

This work was supported in part by the NYUAD Center for Cyber Security (CCS), funded by Tamkeen under the NYUAD Research Institute Award G1104. The authors acknowledge Dr. Amira Guesmi for her contributions, discussions, and feedback on the concept development and methodology based on reinforcement learning, action space refinement, and training optimizations used in this work.

REFERENCES

- [1] James Coker, "Malware Attacks Surge 30% in First Half of 2024," Infosecurity Magazine. Available: <https://www.infosecurity-magazine.com/news/malware-attacks-surge-30-per-cent/>. Accessed: October, 2024.
- [2] AV-Atlas, "Total amount of malware and PUA under Linux," Available: <https://portal.av-atlas.org/malware/statistics>. Accessed: October, 2024.
- [3] Satyajit Sinha, "State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally," IoT Analytics. Available: <https://iot-analytics.com/number-connected-iot-devices/>. Accessed: October, 2024.
- [4] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2014). "Explaining and harnessing adversarial examples". arXiv preprint arXiv:1412.6572.
- [5] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2017). "Towards deep learning models resistant to adversarial attacks". stat, 1050(9).
- [6] Zhan, D., Duan, Y., Hu, Y., Li, W., Guo, S., & Pan, Z. (2023). "MalPatch: Evading DNN-Based Malware Detection With Adversarial Patches". IEEE Transactions on Information Forensics and Security, 19, 1183-1198.
- [7] Li, K., Zhang, F., & Guo, W. (2023). "FGAM: Fast Adversarial Malware Generation Method Based on Gradient Sign". arXiv preprint arXiv:2305.12770.
- [8] Song, W., Li, X., Afroz, S., Garg, D., Kuznetsov, D., & Yin, H. (2022, May). "Mab-malware: A reinforcement learning framework for blackbox generation of adversarial malware". In Proceedings of the 2022 ACM on Asia conference on computer and communications security (pp. 990-1003).
- [9] Zhang, L., Liu, P., Choi, Y. H., & Chen, P. (2022). "Semantics-preserving reinforcement learning attack against graph neural networks for malware detection". IEEE Transactions on Dependable and Secure Computing, 20(2), 1390-1402.
- [10] Shahzad, F., & Farooq, M. (2012). "Elf-miner: Using structural knowledge and data mining methods to detect new (linux) malicious executables". Knowledge and information systems, 30, 589-612.
- [11] Kim, C. W. (2018). "Ntmaldetect: A machine learning approach to malware detection using native api system calls". arXiv preprint arXiv:1802.05412.
- [12] Park, D., Khan, H., & Yener, B. (2019, December). "Generation & evaluation of adversarial examples for malware obfuscation". In 2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA) (pp. 1283-1290). IEEE.
- [13] Khormali, A., Abusnaina, A., Chen, S., Nyang, D., & Mohaisen, A. (2019). "COPYCAT: practical adversarial attacks on visualization-based malware detection". arXiv preprint arXiv:1909.09735.
- [14] Fang, Y., Zeng, Y., Li, B., Liu, L., & Zhang, L. (2020). "DeepDetectNet vs RLAttackNet: An adversarial method to improve deep learning-based static malware detection model". Plos one, 15(4), e0231626.
- [15] Zhong, F., Hu, P., Zhang, G., Li, H., & Cheng, X. (2022). "Reinforcement learning based adversarial malware example generation against black-box detectors". Computers & Security, 121, 102869.
- [16] Etter, B., Hu, J. L., Ebrahimi, M., Li, W., Li, X., & Chen, H. (2023, December). "Evading Deep Learning-Based Malware Detectors via Obfuscation: A Deep Reinforcement Learning Approach". In 2023 IEEE International Conference on Data Mining (ICDM) (pp. 101-109). IEEE.
- [17] Fang, Z., Wang, J., Geng, J., Zhou, Y., & Kan, X. (2021). "A3CMal: Generating adversarial samples to force targeted misclassification by reinforcement learning". Applied Soft Computing, 109, 107505.
- [18] Do Thi Thu, H., Phan, T. D., Le Anh, H., Nguyen Duy, L., Nghi Hoang, K., & Pham, V. H. (2022, December). "A Method of Mutating Windows Malwares using Reinforcement Learning with Functionality Preservation". In Proceedings of the 11th International Symposium on Information and Communication Technology (pp. 142-149).
- [19] Wu, C., Shi, J., Yang, Y., & Li, W. (2018, November). "Enhancing machine learning based malware detection model by reinforcement learning". In Proceedings of the 8th International Conference on Communication and Network Security (pp. 74-78).
- [20] Gibert, D., Fredrikson, M., Mateu, C., Planes, J., & Le, Q. (2022). "Enhancing the insertion of NOP instructions to obfuscate malware via deep reinforcement learning". Computers & Security, 113, 102543.
- [21] Kosikowski, A., Cho, D., Ninan, M., Ralescu, A., & Wang, B. (2023, December). "EvilELF: Evasion Attacks on Deep-Learning Malware Detection over ELF Files". In 2023 International Conference on Machine Learning and Applications (ICMLA) (pp. 1702-1709). IEEE.
- [22] Xue, M., Fu, J., Li, Z., Ni, S., Wu, H., Zhang, L. Y., ... & Liu, W. (2024). "A Reinforcement Learning-based ELF Adversarial Malicious Sample Generation Method". IEEE Journal on Emerging and Selected Topics in Circuits and Systems.
- [23] Qiao, Y., Zhang, W., Tian, Z., Yang, L. T., Liu, Y., & Alazab, M. (2022). "Adversarial ELF malware detection method using model interpretation". IEEE Transactions on Industrial Informatics, 19(1), 605-615.
- [24] Ravi, A., Chaturvedi, V., & Shafique, M. (2023). "ViT4Mal: Lightweight Vision Transformer for Malware Detection on Edge Devices". ACM Transactions on Embedded Computing Systems, 22(5s), 1-26.
- [25] Khan, R. U., Zhang, X., & Kumar, R. (2019). "Analysis of ResNet and GoogleNet models for malware detection". Journal of Computer Virology and Hacking Techniques, 15, 29-37.
- [26] Manavi, F., & Hamzeh, A. (2020, September). "A new method for ransomware detection based on PE header using convolutional neural networks". In 2020 17th international ISC conference on information security and cryptology (ISCISC) (pp. 82-87). IEEE.
- [27] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). "Grad-cam: Visual explanations from deep networks via gradient-based localization". In Proceedings of the IEEE international conference on computer vision (pp. 618-626).
- [28] bogwonch, "Semantic-NOPs," GitHub repository, 2011. [Online]. Available: <https://github.com/bogwonch/Semantic-NOPs>.
- [29] AlMahamid, F., & Grolinger, K. (2021, September). "Reinforcement learning algorithms: An overview and classification". In 2021 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE) (pp. 1-7). IEEE.
- [30] Romain Thomas, "LIEF-Library to Instrument Executable Formats." Available: <https://lief.quarkslab.com/>. Accessed: April, 2024.
- [31] VirusTotal, Available: <https://www.virustotal.com/>. Accessed: August, 2024.
- [32] Pa, Y. M. P., Suzuki, S., Yoshioka, K., Matsumoto, T., Kasama, T., & Rossow, C. (2016). "IoTPOT: A novel honeypot for revealing current IoT threats". Journal of Information Processing, 24(3), 522-533.
- [33] Anderson, H. S., Kharkar, A., Filar, B., Evans, D., & Roth, P. (2018). "Learning to evade static pe machine learning malware models via reinforcement learning". arXiv preprint arXiv:1801.08917.